1 2  9 0

UNIVERSIDADE D
COIMBRA

Automated Software Engineering

Course Project Report • 2025/2026

FilmHub

A Movie Recommendation System:
Traditional vs. AI-Assisted Development

Group Members:

Diogo Martins	2021216461	uc2021216461@student.uc.pt
Eduardo Marques	2022231584	uc2022231584@student.uc.pt
Gonçalo Nogueira	2022213646	uc2022213646@student.uc.pt
João Cardoso	2022222301	uc2022222301@student.uc.pt
Martin Resplandy	2025155106	uc2025155106@student.uc.pt
Pierre Tallec	2025155117	uc2025155117@student.uc.pt

December 14, 2025

Abstract

This report details the development of "FilmHub," a movie recommendation system, created as part of the Automated Software Engineering course. The project explores two distinct development methodologies: a traditional manual workflow and an AI-assisted workflow utilizing Large Language Models (LLMs). The report compares these approaches in terms of architecture, development velocity, code quality, workflow and llm vs no llm usage. It also covers the implementation of DevOps practices, including CI/CD pipelines and containerization.

Contents

Chapter 1

Introduction

1.1 Project Context

This project was developed as part of the Automated Software Engineering course for the 2025/2026 academic year. The primary goal is to provide hands-on experience with modern software development practices by implementing a complete recommendation system while exploring two distinct development approaches: traditional manual development and AI-assisted development using Large Language Models (LLMs).

The project covers the full Software Development Lifecycle (SDLC), from initial requirements gathering and system design through implementation, testing, deployment, and monitoring. A key aspect of this work is the integration of DevOps practices through the configuration and use of Continuous Integration and Continuous Delivery/Deployment (CI/CD) pipelines, which automate critical workflow stages such as testing, quality assurance, and deployment.

By comparing traditional and AI-assisted development methodologies, this project aims to provide empirical insights into the effectiveness, efficiency, and quality implications of incorporating LLM-based tools into professional software engineering workflows. This comparative analysis is supported by quantitative metrics and qualitative observations documented throughout the development process.

1.2 System Overview: FilmHub

FilmHub is a comprehensive movie recommendation system designed to help users discover films based on their preferences and viewing history. The system provides personalized movie suggestions by analyzing user ratings and leveraging the TMDb (The Movie Database) API to access a vast catalog of films with detailed metadata.

The application consists of three main architectural components:

Backend Service: Built with Django and Django REST Framework, the backend provides a RESTful API that manages all business logic, data persistence, and integration with external services. It implements authentication using token-based mechanisms, handles user management, processes movie ratings, and executes the recommendation algorithm based on genre and keyword matching.

Database Layer: The system uses PostgreSQL as its relational database to store users, movies, ratings, and user profiles. The database schema is designed to support efficient queries for recommendation generation while maintaining data integrity through

constraints and relationships.

Frontend Application: Developed with React, the frontend provides an intuitive and responsive user interface. Users can browse movie catalogs organized by categories (Popular, Top Rated, Action, Comedy, Drama), search for specific films by title, director, or genre, rate movies on a scale of 1–10, view personalized recommendations, and manage their viewing history.

The recommendation engine operates by analyzing user ratings to identify preferred genres and keywords. It assigns weighted scores to potential recommendations, prioritizing movies that match multiple user preferences while excluding films already watched or rated. The system integrates seamlessly with the TMDb API to dynamically fetch movie details, including posters, descriptions, cast information, and ratings.

1.3 Objectives

The project is structured around several interconnected objectives that align with modern software engineering principles and practices:

1. **Experience the Full SDLC**: Implement all phases of software development, from requirements analysis and system design through coding, testing, deployment, and maintenance, providing comprehensive exposure to professional development workflows.

2. **Compare Development Methodologies**: Conduct a systematic comparison between traditional manual development and AI-assisted development using LLMs, evaluating factors such as development speed, code quality, bug frequency, testing coverage, and developer productivity.

3. **Apply DevOps Principles**: Implement industry-standard DevOps practices by configuring CI/CD pipelines that automate building, testing, and deployment processes, reducing manual intervention and improving software delivery reliability.

4. **Configure CI/CD Pipelines**: Set up automated pipelines using tools such as GitHub Actions, GitLab CI, Jenkins, or Azure DevOps to support continuous integration, automated testing, and continuous deployment to production or staging environments.

5. **Develop Autonomy and Self-Learning Skills**: Foster independent problem-solving abilities and research skills by navigating technical challenges, exploring documentation, and adapting to new technologies and methodologies throughout the project lifecycle.

6. **Implement a Functional Recommendation System**: Deliver a working application that provides meaningful value to users through accurate movie recommendations, intuitive user experience, and reliable performance under typical usage conditions.

7. **Deploy to Production Environment**: Successfully deploy the complete system using containerization (Docker) or cloud-based infrastructure, implementing appropriate deployment patterns, monitoring solutions, and ensuring system availability and scalability.

8. **Document and Reflect**: Produce comprehensive documentation covering system architecture, implementation decisions, comparative analysis results, and lessons learned, providing insights that can inform future development projects and contribute to the broader understanding of AI-assisted development practices.

These objectives collectively ensure that the project not only results in a functional application but also provides valuable learning experiences in contemporary software engineering practices, preparing students for professional development environments where automation, quality assurance, and efficient workflows are paramount.

Chapter 2

Implementation and Source Code

This project was developed in two distinct phases, each with its own dedicated repository to maintain clear separation between the traditional machine learning approach and the large language model (LLM) implementation.

2.1 Repository Organization

The implementation of this project is organized across two GitHub repositories:

- **Traditional Approach Repository**: The classical machine learning implementation, including data preprocessing, feature engineering, and traditional recommendation algorithms, was developed in the first repository available at:

  `https://github.com/GoncaloNogueira1/filmhub-project`

- **LLM-Based Approach Repository**: The large language model implementation, including prompt engineering, model integration, and LLM-powered recommendation features, was developed in the second repository available at:

  `https://github.com/GoncaloNogueira1/filmhub-llm`

This separation allows for independent development and testing of both approaches, facilitating comparison and evaluation of the different methodologies employed in the film recommendation system.
Deployed Applications: Links of the apps online:
Traditional Approach: `https://filmhub-frontend-n6i3.onrender.com`
LLM-Based Approach: `https://filmhub-frontend-llm.onrender.com`

2.2 Repository Structure

Both repositories contain comprehensive documentation, source code, and relevant resources necessary for understanding and reproducing the work presented in this thesis. Detailed README files in each repository provide specific instructions for setup, configuration, and execution of the respective implementations.

Chapter 3

Requirements and Specifications

3.1 Functional Requirements (Use Cases)

The functionality of FilmHub was defined through 11 specific Use Cases. These requirements apply to both the Traditional and AI-Assisted versions of the system. Initially, we generated the requirements also using AI, but we found that they were overly complex and unnecessary. As a result, we decided to use requirements that are almost the same as those of the traditional version (with the differences reported later).

3.1.1 Authentication and User Management

User Registration: New users can create accounts by providing username, email, and password. The system validates password strength requiring minimum 8 characters with letters, numbers, and special characters. Username and email uniqueness is enforced at database level.

User Login: Registered users authenticate using username and password credentials. Upon successful authentication, the system issues a JWT token for subsequent API requests. Invalid credentials return appropriate error messages.

Session Management: Authentication tokens persist in browser localStorage enabling automatic login on page refresh. Users can manually logout which clears tokens and redirects to the login page.

3.1.2 Movie Browsing and Discovery

Catalog Browsing: Users access organized movie catalog with sections for Popular Movies, Top Rated, Action, Comedy, and Drama. Each section displays up to 20 movies with poster, title, year, genre, and average rating.

Movie Search: Users search movies by title, director, or genre. Search queries are processed by the backend which fetches results from TMDB API. Results display in consistent card grid format with empty states for no matches.

Movie Details: Users view comprehensive movie information including large poster, full description, release year, genre, average rating with vote count, and personal rating status.

3.1.3 Rating System

Submit Ratings: Users rate movies on a scale of 1 to 10 stars through modal dialog interface. Ratings are stored with an optional comment field. Each user can rate each movie only once.

Update Ratings: Users modify existing ratings by opening rating modal on previously rated movies. Updated ratings immediately reflect in the UI and trigger recommendation refresh.

View Personal Ratings: Users access centralized page displaying all their rated movies with full details and current ratings. This page enables easy tracking of rating history.

3.1.4 Recommendation Engine

Personalized Recommendations: System generates up to 20 movie recommendations based on user rating patterns. Algorithm analyzes genres and keywords from highly-rated movies (score 6 or above) and discovers similar content via TMDB API.

Recommendation Refresh: Recommendations automatically update when users submit or modify ratings. The system excludes already-rated, watched, or watchlisted movies from recommendations.

3.1.5 Additional Features (AI-Assisted Version)

Watch List Management: Users add movies to the watch list for later viewing. The system prevents duplicates and provides endpoints to view, add, and remove movies from the watch list.

Watched Movies Tracking: Users mark movies as watched. The system maintains a separate collection of watched movies accessible through dedicated endpoints.

3.1.6 LLM-Assisted Version Differences

While both implementations share the same core functional requirements, the LLM-Assisted version includes several differences and additional features:

- **Rating Scale:** Uses a 1-5 star rating system (compared to 1-10 in the Traditional version)
- **Authentication:** Implements JWT (JSON Web Tokens) authentication instead of Django REST Framework's Token Authentication
- **User Profile Management:** Extended user profiles with additional information fields and dedicated profile management endpoints
- **Database Population Strategy:** The database is pre-populated with approximately 300 movies during deployment using the `import_tmdb_movies` management command, and the movie catalog is served directly from the database. In contrast, the Traditional version fetches the movie catalog directly from the TMDB API and only stores movies in the database on-demand when users rate them or add them to their watchlist.

3.2 Non-Functional Requirements

3.2.1 Performance

Response Time: API endpoints respond within 3 seconds under normal load. Movie catalog and search operations complete within 3 seconds including external API calls.

Concurrent Users: System supports at least 50 concurrent users without performance degradation. Database connection pooling handles simultaneous requests efficiently.

External API Integration: TMDB API calls implement 5-second timeout to prevent hanging requests. Failed API calls gracefully degrade with cached data or appropriate error messages.

3.2.2 Security

Authentication: Token-based authentication secures all protected endpoints. Tokens are required in the Authorization header for authenticated operations.

Password Storage: User passwords are hashed using Django's default PBKDF2 algorithm with SHA256. Plain text passwords are never stored in a database.

Input Validation: All user inputs validated server-side to prevent injection attacks. Email format, password strength, and rating ranges enforced through Django validators.

CORS Configuration: Cross-Origin Resource Sharing restricted to localhost:3000 for development. Production deployment requires proper CORS configuration for the frontend domain.

3.2.3 Reliability

Error Handling: Comprehensive try-catch blocks wrap all async operations. Errors logged for debugging and converted to user-friendly messages for display.

Database Constraints: Unique constraints on username, email, and movie combinations prevent duplicate entries. Foreign key constraints maintain referential integrity.

Data Consistency: Rating operations check for existing entries before creation. Update operations verify existence before modification preventing orphaned data.

3.2.4 Usability

Responsive Design: Application adapts to mobile, tablet, and desktop screen sizes. Touch targets meet 44x44 pixel minimum for mobile accessibility.

Loading Indicators: Clear loading states during data fetches prevent user confusion. Spinners and disabled buttons indicate ongoing operations.

Error Feedback: Specific error messages guide users to resolve issues. Validation errors highlight problematic fields with actionable instructions.

3.2.5 Scalability

Database Design: PostgreSQL database with indexed foreign keys enables efficient queries. Many-to-many relationships through intermediate tables support growing data volumes.

API Architecture: RESTful design with clear resource separation enables horizontal scaling. Stateless authentication simplifies load balancing.

Caching Strategy: External API responses could be cached to reduce redundant TMDB requests. Movie data stored locally after first fetch eliminates repeated external calls.

3.3 Architecture: Client-Server Monolith

The system implements a client-server architecture with clear separation between frontend React application and backend Django REST API. Communication occurs over HTTP using JSON payloads.

3.3.1 Architecture Pattern

The application follows a three-tier monolithic architecture:

Presentation Layer: React SPA running in browser handles all UI rendering and user interactions. Components manage local state and communicate with backend through service layer.

Application Layer: Django REST Framework processes HTTP requests, executes business logic, manages authentication, and coordinates between presentation and data layers.

Data Layer: PostgreSQL relational database stores persistent data including users, movies, ratings, and user profiles with relationships. Django ORM abstracts database operations.

3.3.2 Communication Flow

Client-server communication follows RESTful principles with clear resource endpoints. Frontend initiates requests with authentication tokens. Backend validates tokens, processes requests, interacts with database or external APIs, and returns JSON responses. Frontend updates UI based on responses.

3.3.3 Technology Stack

Frontend Technologies **React 18:** JavaScript library for building user interfaces using component-based architecture. Hooks enable functional components with state and effects.

React Router v6: Client-side routing library managing navigation between pages without full page reloads. Implements protected routes through the RequireAuth component.

CSS3: Styling with modern features including Grid, Flexbox, custom properties, and animations. Component-specific stylesheets co-located with components.

Fetch API: Native browser API for HTTP requests to backend. No external HTTP libraries like Axios required.

localStorage: Browser storage API for persisting authentication tokens between sessions.

Backend Technologies **Django 5.2.8:** Python web framework providing ORM, authentication, admin interface, and URL routing. Follows MVT (Model-View-Template) pattern adapted for API.

Django REST Framework: Toolkit for building Web APIs with serializers, viewsets, authentication, and permissions. Simplifies API endpoint creation.

PostgreSQL: Relational database management system storing structured data with ACID compliance. Supports complex queries and relationships.

Token Authentication: Django REST Framework's token authentication provides a stateless authentication mechanism. Each user receives a unique token on login.

External Services **TMDB API:** The Movie Database API provides comprehensive movie information including titles, descriptions, posters, genres, keywords, ratings, and cast details. System fetches movie data on-demand and stores locally.

Development Tools **Jest:** JavaScript testing framework for unit tests. React Testing Library provides utilities for testing components from user perspective.

Django Test Framework: Built-in testing framework for backend unit and integration tests. Supports fixtures, mocking, and database isolation.

Deployment Stack The project is designed for containerized deployment though current development uses local servers:

Frontend: React development server on port 3000 for development. Production build generates static files for serving through Nginx or CDN.

Backend: Django development server on port 8000. Production deployment uses Gunicorn WSGI server behind Nginx reverse proxy.

Database: PostgreSQL server on port 5432. Connection pooling configured through Django database settings.

3.3.4 Data Flow

User Authentication Flow The user submits credentials through the login form. Frontend sends POST requests to `/api/login/` endpoint with username and password. Backend authenticates using Django's authenticate function. If valid, the backend retrieves or creates authentication token and returns token with user object. Frontend stores the token in localStorage and AuthContext. Subsequent requests include a token in the Authorization header as "Token <token_key>". Backend validates token on each protected endpoint request using TokenAuthentication class.

Movie Browsing Flow The user navigates to the home page triggering catalog load. Frontend sends GET requests to `/api/movies/` with authentication token. Backend checks if the search parameter exists. If no search, the backend makes parallel requests to the TMDB API for popular, top rated, and genre-specific movies. Backend formats responses using `format_movie` utility function extracting relevant fields. Backend returns catalog objects with categorized movie arrays. Frontend displays movies in organized sections using MovieList and MovieCard components.

Movie Search Flow The user enters a search query and clicks the search button. Frontend sends GET requests to `/api/movies/?search=query&search_type=title` with token. Backend receives search and `search_type` parameters. Based on `search_type` (title, director, or genre), backend calls appropriate TMDB API endpoints. For director search, backend first searches the person endpoint then retrieves their movie credits. For genre search, backend maps genre name to ID and uses discover endpoint. Backend formats results and returns movie array. Frontend displays search results replacing catalog view.

Rating Submission Flow User clicks Rate This Movie button opening rating modal. The user selects ratings from 1-10 and clicks Submit. Frontend sends POST requests to `/api/ratings/` with movie `external_id`, score, and optional comment. The backend checks if the movie exists in the local database. If it does not exist, backend calls `create_movie_from_external_id` which fetches from TMDB API and stores locally. Backend verifies the user hasn't already rated this movie. Backend creates a Rating object linking user, movie, and score. Backend triggers `update_recommendations` for user profile. Backend returns created rating object. Frontend displays updated rating in UI and closes modal.

Rating Update Flow User clicks Update Rating on previously rated movie. Frontend pre-selects existing ratings in modal. The user modifies rating and submits. Frontend sends PATCH requests to `/api/ratings/` with movie `external_id` and new score. Backend locates existing Rating objects for user and movie. Backend updates score and comment fields. Backend triggers recommendation refresh. Backend returns updated rating object. Frontend reflects changes throughout the application.

Recommendation Generation Flow Triggered automatically after rating submission or update. Backend retrieves all user ratings from the database. Backend filters ratings with score 6 or above identifying liked movies. Backend extracts genres and keywords from liked movies building preference profile. Backend converts genre and keyword names to TMDB API IDs using mapping dictionaries. Backend makes discovery requests to TMDB for movies matching genres (1 point per match) and keywords (3 points per match). Backend accumulates points for each recommended movie and sorts by score. Backend filters out movies users have already rated, watched, or added to the watch list. Backend creates or retrieves Movie objects for top 20 recommendations. Backend adds movies to UserProfile `recommended_movies` many-to-many relationship. Frontend fetches recommendations via GET `/api/recommended_movies/` displaying in a dedicated page.

Database Interaction Flow All database operations occur through Django ORM. Backend views call utility functions which execute ORM queries. Create operations using `Model.objects.create()` or `serializer.save()`. Read operations use `Model.objects.filter()` or `get()` with query parameters. Update operations retrieve objects, modify fields, and call `save()`. Delete operations use `Model.objects.filter().delete()`. Many-to-many operations use `add()`, `remove()`, and `clear()` on relationship managers. Database constraints automatically enforce uniqueness and foreign key integrity. Failed constraint violations raise exceptions caught by views and returned as error responses.

External API Integration Flow Backend makes requests to TMDB API when local data is unavailable. Requests include API key in query parameters for authentication. Responses contain comprehensive movie data in JSON format. Backend extracts relevant fields (title, `poster_path`, overview, genres, `release_date`, runtime). Backend transforms data to match internal Movie model schema. Genre IDs converted to names using `GENRE_MAP` dictionary. Poster paths converted to full URLs with image server base. Release dates parsed to extract year. Backend creates Movie objects with transformed data. Errors during API requests caught and logged without crashing applications. Failed requests return None allowing graceful degradation.

Chapter 4

Process & Autonomy

4.1 Team Organization and Task Distribution

4.1.1 Team Roles

The team was organized around primary roles to promote ownership and accountability. In practice, however, the project required frequent collaboration across roles. While each member had a main responsibility, development tasks often overlapped and were carried out collaboratively through shared debugging, code reviews, and joint decision-making.

Table 4.1: Team Member Roles and Responsibilities

Team Member	**Primary Role**
Eduardo Marques	Frontend (Traditional)
Gonçalo Nogueira	Backend (Traditional), Backend and Frontend (LLM)
João Cardoso	Frontend (Traditional)
Martin Resplandy	Backend (Traditional)
Pierre Tallec	Backend (Traditional)
Diogo Martins	Research

4.1.2 Task Distribution

The team maintained effective communication and coordination through clearly defined milestones and collaborative tools. The project progress was structured as follows:

- **Phase 1: Research and Requirements Analysis (Milestone 1)** Initial research on recommendation systems and relevant technologies, followed by the definition of functional and non-functional requirements. This phase established the scope of the project and served as the foundation for both development approaches.
- **Phase 2: Project Setup and Version Control (Milestone 2)** Creation of GitHub repositories, definition of the project structure, and setup of version control workflows. Initial backend scaffolding and early CI/CD pipeline configuration were also introduced during this phase.
- **Phase 3: Frontend Design and Mockups** Design of user interface mockups and development of initial frontend components, defining user flows, layout structure, and visual identity.

- **Phase 4: Backend Development** Implementation of backend functionality, including API endpoints, database models, authentication mechanisms, and recommendation logic.
- **Phase 5: System Integration (Traditional)** Integration of frontend and backend components for the traditional.
- **Phase 6: Full devolpment LLM (Backend + Frontend)** LLM full development
- **Phase 7: Debugging, Stabilization, and Deployment** Extensive debugging and validation to ensure system correctness, followed by the deployment of both versions of the application and final reviews of the CI/CD pipelines.
- **Phase 8: Reporting and Documentation** Preparation of detailed documentation, comparative analysis between development approaches, and writing of the final project report.

4.1.3 Communication

Effective communication was maintained through multiple channels:

- **WhatsApp Group and private messages:** Used for daily updates, quick problem-solving, and immediate team coordination.
- **Weekly Team Meetings:** Held to discuss progress, address project challenges, and plan upcoming tasks. Meetings were conducted during Friday classes, and occasionally via Discord full team or just backend or frontend team.
- **GitHub:** Served as the primary platform for code repository management, version control, and issue tracking.

4.2 Version Control Practices

4.2.1 Repository Structure

Two separate repositories were maintained for clear comparison between approaches:

- `filmhub-project/` - Traditional implementation
- `filmhub-llm/` - AI-assisted implementation

4.2.2 Git Workflow

Branching Strategy:

- `main`: Principal shared branch
- Personal branches for individual or small group work

Commit Standards: The team adopted Conventional Commits standards for consistency:

- Traditional repository: 276 commits, 20 pull requests
- AI-Assisted repository: 26 commits, 0 pull requests (since it was mainly done by one person, there is no need to pull request review)
- All pull requests required peer review before merge

4.3 Self-Learning and Autonomy

4.3.1 Learning Resources

Although the team already had prior experience with several of the technologies used, the project required deeper understanding and practical application in new contexts. As a result, team members actively sought additional learning resources and independently addressed knowledge gaps as they emerged during development.

Backend Development:

- Django and Django REST Framework: Official documentation and targeted youtube tutorials were consulted to clarify advanced topics such as authentication, serializers, and API design.
- LLM Techniques: Prompt engineering strategies were explored using course slides, online research, and reference to a student research projects, with a focus on improving reliability and code quality.

Frontend Development:

- React: Official documentation and tutorials were used to refine understanding of hooks, routing, and component composition, occasionally supported by LLM tools.
- State Management: Practical use of React Context API and Zustand to manage global application state effectively.

DevOps and Infrastructure:

- Docker: Learning focused on containerizing multi-service applications and managing development and production configurations.
- GitHub Actions: Configuration of CI/CD workflows for automated testing, validation, and deployment.
- Cloud Deployment: Exploration of basic cloud deployment strategies suitable for containerized applications.

This self-learning process strengthened the team's autonomy and enabled members to independently solve technical challenges as they emerged throughout the project.

4.4 Development Metrics

4.4.1 Development Velocity

The team's productivity varied over the course of the project, reflecting different phases of work:

- **25 September – 15 October:** The initial phase focused on research, discussions, and milestones. Progress was slow as the team worked to understand project requirements.
- **16 October – 25 November:** The team began implementing initial solutions while continuing discussions and research. Progress was steady but still gradual due to ongoing planning and adjustments.
- **25 November – 12 December:** Development intensified significantly. The team worked at a high pace to finalize the project.

This progression reflects the nature of the work, with early phases dedicated to planning and research, and the final phase emphasizing focused execution and concentrated effort.

4.4.2 Development Effort Distribution

Table 4.2 presents the total estimated effort invested by each team member throughout the project lifecycle, encompassing both traditional and AI-assisted development phases, including weekly practical lab sessions (PL). **Effort Distribution Notes:**

Table 4.2: Total Development Effort by Team Member

Team Member	Development+Report	PL Sessions+Meetings	Total Hours
Eduardo Marques	70h	20h	90h
Gonçalo Nogueira	70h	20h	90h
João Cardoso	70h	20h	90h
Martin Resplandy	70h	20h	90h
Pierre Tallec	70h	20h	90h
Diogo Martins	0h	20h	20h
Total Team Effort	**350h**	**120h**	**470h**

- **Development + Report Hours**: Time spent on active coding, testing, debugging, documentation, and deployment activities outside of scheduled lab sessions.
- **PL Sessions**: All team members attended weekly 2-hour practical lab sessions throughout the semester (approximately 10 sessions × 2h = 20h per member), where project progress was discussed, issues were addressed, and team coordination occurred.
- **Total team effort** represents 470 person-hours across the 3-month project timeline (mid-September to mid-December), averaging approximately 78.3 hours per team member, or approximately 6.5 hours per week including lab sessions.

- **Effort breakdown by approach**:
  - Traditional implementation: approximately 320h (shared across 5 members)
  - LLM-assisted implementation: approximately 30h (primarily Gonçalo)
  - Research: approximately 20h (primarily Diogo)
  - Total: 350h development + 120h PL sessions = 470h

4.5 Reflection

4.5.1 What Worked Well

- Clear role definition with flexibility for cross-functional collaboration
- Strong communication through WhatsApp and regular team meetings
- Systematic issue tracking via GitHub
- Proactive problem-solving approach
- Effective cross-team collaboration

4.5.2 What We'd Improve

- More detailed initial time estimates for better planning
- Earlier integration testing

4.5.3 Skills Developed

Technical Skills:

- Full-stack development (Django/React/PostgreSQL)
- DevOps practices (Docker/CI-CD)
- AI-assisted development techniques
- Software testing methodologies

Soft Skills:

- Team collaboration and communication
- Project management
- Self-directed learning
- Technical writing and documentation

Process Skills:

- Agile development methodologies

- Git workflow and version control
- CI/CD pipeline management
- Quality assurance practices

Chapter 5

Traditional Development Workflow

5.1 Methodology and Approach

The primary objective of this phase was to implement the FilmHub system strictly adhering to a traditional Software Development Lifecycle (SDLC). This served as a control baseline to later evaluate the efficiency and impact of AI-assisted tools.

The development process followed a standard iterative approach, characterized by manual execution of all engineering tasks:

- **Manual Coding:** All source code, including boilerplate structures for the Django backend and React frontend, was written manually by the development team. Some were already familiar with the stack, the others watched Youtube videos to learn.
- **Human-Driven Debugging:** Error resolution and code refactoring were conducted with very little aid of Large Language Models (LLMs) or automated code generation tools.
- **Documentation:** Technical documentation, including API specifications and setup guides, was authored manually based on the system requirements. We used a Postman collection to share the routes between the backend team and the frontend team.

5.2 Implementation Strategy

While the architectural components (React, Django, PostgreSQL) were defined in Chapter 2, the implementation workflow presented specific challenges inherent to the traditional approach.

5.2.1 Backend Integration

Technologies and Stack Selection

The backend was developed using a robust, community-supported stack chosen for its scalability, rapid development capabilities, and strong integration ecosystem.

Table 5.1: Selected Backend Stack Components

Component	Technology	Rationale
Framework	Django	Selected for its powerful ORM, built-in security features, and extensive documentation, which allowed for quick iteration on core features.
API	Django REST Framework (DRF)	Used for building a flexible and consistent RESTful API interface, enabling seamless communication with the frontend application. DRF provided serialization, viewsets, and routing capabilities.
Database	PostgreSQL	Chosen over SQLite for production use due to its reliability, advanced indexing capabilities, and robust support for transactional data integrity and concurrency.
Authentication	Django REST Framework SimpleJWT	Implemented for secure, stateless user authentication using JSON Web Tokens (JWTs), allowing the frontend to securely access protected endpoints.
Testing	`pytest` & Django's `TestCase`	Utilized for unit and integration testing to ensure code reliability and prevent regressions during feature development.

Architectural Design: Layered Structure

The backend follows a layered architectural pattern to ensure separation of concerns, maintainability, and testability.

- **Presentation Layer (Views/ViewSets):** Handled incoming HTTP requests, delegated tasks to the Business Logic layer, and returned standardized HTTP responses. This layer primarily uses DRF ViewSets.
- **Business Logic Layer (Service/Utility Functions):** Contains the core logic of the application (e.g., calculating movie ratings, managing user permissions, validation). This was implemented using dedicated service functions (as seen in the rating handlers) to keep the views thin.
- **Data Access Layer (Models/Managers):** Manages all interaction with the

PostgreSQL database via the Django ORM. This layer defines the data structures (`Movie`, `User`, `Rating`, etc.) and handles data persistence.

```
filmhub-project/
          api/                        # Single Django app
                models.py              # All models (Movie, Rating, etc
   )
                views.py               # All views
                serializers.py         # All serializers
                utils.py               # Business logic
                validators/            # Input validation
                      shared.py         # Common validators
                      normal.py         # Movie validators
                admin.py               # Admin configuration
                urls.py                # API routing
                tests.py               # Single test file (22 tests)
```

Listing 5.1: Traditional Backend Structure

Key Backend Functionality

The following key features were implemented and exposed via the REST API:

1. **Data Modeling and Relations**

   The core functionality revolved around defining the relationships between the primary data entities:

   - **One-to-Many:** One user can create multiple ratings.
   - **Many-to-Many:** Implied through the `Rating` model, which connects `User` and `Movie`.
   - **External Data Integration:** The `Movie` model included an `external_id` field to maintain a link to external movie databases, ensuring data consistency and enabling on-demand data fetching.

2. **User Authentication and Authorization**

   - **Registration/Login:** Implemented standard user registration and secure login via JWT token generation.
   - **Protected Endpoints:** All critical endpoints (e.g., `POST` a rating, `PUT` an existing rating) required a valid JWT in the request header to ensure only authenticated users could modify data.

3. **Rating and Review Management** The backend implemented comprehensive logic for creating, retrieving, updating, and deleting ratings, including necessary validation (e.g., ensuring rating scores are within a defined range).

Challenges and Solutions

Developing the backend presented several technical challenges that required careful design and implementation decisions:

- **Challenge 1: Ensuring Data Integrity and Concurrency**
  - **Problem:** When checking if a rating already exists and then creating a new one (a "check-then-act" operation) in the `add_rating` function, a race condition could occur if two users attempted the same action simultaneously.
  - **Solution:** We utilized Django's database transactions and, specifically, atomic transactions (`@transaction.atomic`) around the `Rating.objects.get` and `Rating.objects.create` block. This ensured that the entire operation either succeeds completely or fails entirely, preventing duplicate ratings.
- **Challenge 2: Handling External API Integration Failures**
  - **Problem:** The `create_movie_from_external_id` function, which fetches movie details from a third-party API, is subject to external network latency or failure (e.g., a 500 error from the external service).
  - **Solution:** Implemented robust error handling using `try...except` blocks with specific exceptions (e.g., `requests.exceptions.RequestException`). If the external API call failed, the service function returned a descriptive status (`Status.EXTERNAL_FAILURE`) rather than crashing, ensuring the core application remained stable.
- **Challenge 3: Maintaining Clean Business Logic in DRF**
  - **Problem:** Initially, too much business logic (like the existence check in `add_rating`) was being written directly into DRF views or serializers, making them bloated and difficult to test independently.
  - **Solution:** Adopted the Service/Business Logic Pattern. Dedicated service functions (like `add_rating` and `update_rating`) were created to handle all the core business rules. The DRF serializers and views were then primarily responsible for data format and validation, calling the service layer to perform the actual work.

5.2.2 Frontend Design and Mockups

Before beginning the actual implementation, the frontend team developed mockups to establish the visual design and user experience of FilmHub. This design-first approach allowed us to:

- Define the overall visual identity and user interface style
- Establish consistent layout patterns across different pages
- Plan user flows and interactions before writing code
- Facilitate communication between team members about design decisions
- Serve as a reference during implementation

Key Mockup Screens

The mockup phase focused on three core interfaces that represent the primary user journey:

1. **Login Page:** Clean, centered authentication form with the FilmHub branding and a minimalist dark theme design. This established the application's visual identity with a focus on simplicity and usability.

2. **Main Catalog Page:** Grid-based movie catalog layout displaying movie posters, titles, ratings, and a search bar. The design emphasized visual hierarchy with the welcome message, search functionality, and organized movie sections (Popular Movies, Top Rated, etc.).

3. **Movie Details Page:** Detailed view showing enlarged movie poster, comprehensive movie information (title, year, genre, runtime), rating display with star visualization, and prominent "Rate This Movie" action button.

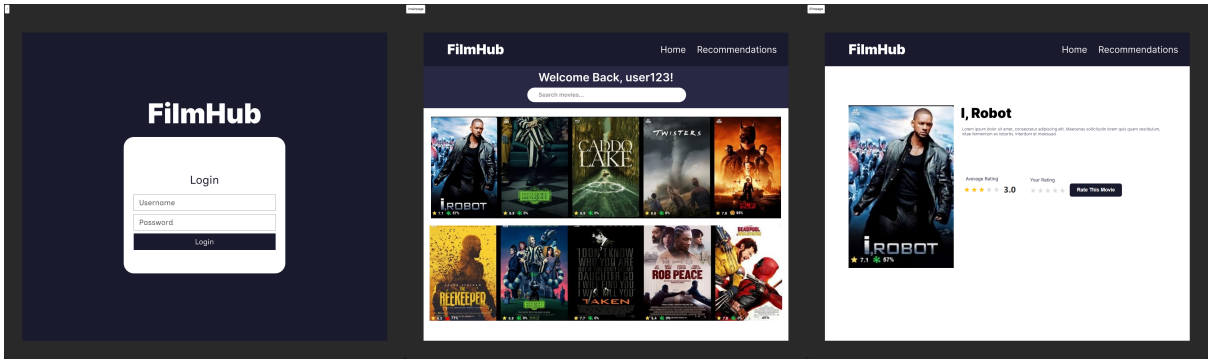

Figure 5.1: Frontend Mockup

Design Decisions

Several key design decisions emerged from the mockup phase:

- **Dark Theme:** A dark navy blue background (#1a1d29 or similar) was chosen to create a good eye-catching atmosphere.

- **Visual Hierarchy:** Movie posters serve as the primary visual elements, with clear typography hierarchy for titles, metadata, and ratings.

- **Star Rating System:** Visual star icons for ratings provide immediate, intuitive feedback about movie quality, with both average ratings and user-specific ratings displayed.

- **Responsive Grid Layout:** The catalog uses a flexible grid system that adapts to different screen sizes, ensuring a consistent experience across devices.

- **Consistent Navigation:** A fixed navigation bar provides easy access to key features (Home, Recommendations, User Menu) throughout the application.

From Mockups to Implementation

The mockups served as a blueprint during the implementation phase, though some adjustments were made during development:

- Component structure was refined to maximize reusability (e.g., MovieCard component used across multiple pages)
- Interaction patterns were enhanced with hover effects and smooth transitions
- Responsive behavior was fine-tuned based on testing across different viewport sizes
- Color palette was slightly adjusted for better contrast and accessibility

Having these mockups significantly streamlined the development process by providing clear visual targets and reducing design-related discussions during implementation.

5.2.3 Frontend Integration

Application Structure

The application follows a component-based architecture organized into several distinct layers:

- **Components:** Reusable UI elements including Layout, Navbar, MovieCard, MovieList, SearchBar, RequireAuth, and MovieDetails. Each component encapsulates specific functionality and styling.
- **Pages:** Top-level views for Auth (login/register), MainPage (catalog), MovieDetails, Recommendations, and Ratings.
- **Services:** API communication layer abstracting HTTP requests. The `authService` handles authentication while `movieService` manages movie data and ratings.
- **Context:** AuthProvider manages authentication state application-wide using React Context API.
- **Hooks:** Custom `useAuth` hook provides convenient authentication state access and logout functionality.

```
frontend/src/
          components/             # 7 components
                Layout/
                MovieCard/
                MovieDetails/
                MovieList/
                Navbar/
                RequireAuth/
                SearchBar/
          pages/                  # 5 pages
                Auth/
                MainPage/
                MovieDetails/
                Ratings/
                Recommendations/
          services/               # 2 API services
                authService.js
                movieService.js
          context/                # React Context
                AuthProvider.jsx
          hooks/
                useAuth.jsx
          config.js              # API configuration
```

Listing 5.2: Traditional Frontend Structure

Routing and Protection

React Router v6 implements protected routing with public routes for login/register and protected routes requiring authentication for home, movie details, recommendations, and ratings pages. The RequireAuth component guards routes, redirecting unauthenticated users to login while preserving intended destinations.

State Management

Authentication state is managed globally through AuthContext, persisting across refreshes via localStorage synchronization. Local component state handles UI interactions like loading indicators, error messages, form inputs, and modal visibility. The `useAuth` hook prevents prop drilling by providing clean authentication access.

Data Flow

Unidirectional data flow follows React patterns: user interactions trigger event handlers, service functions make authenticated HTTP requests, responses update component state triggering re-renders. Authentication tokens from localStorage are automatically included in API requests through `getAuthHeaders`.

Key Features Implementation

Authentication System: Complete user management with registration (username, email, password), login with JWT tokens, and session persistence through localStorage. The Auth component toggles between login/registration modes. On app load, existing tokens restore automatically, preventing unnecessary re-authentication.

Movie Catalog: Organized catalog displays Popular Movies, Top Rated, Action, Comedy, Drama, and All Movies sections. MovieCard components show posters (with fallback placeholders), titles, year, genre, average ratings as stars, and user's personal rating. Hover animations and click navigation to details. Responsive grid layouts adapt to viewport width.

Search: SearchBar provides clean interface with icon, input, clear button, and submit. Searches call movieService with trimmed queries. Results display in MovieList format with empty states for no matches. Clear button returns to catalog view.

Movie Details: Dynamic routing displays comprehensive movie information: large poster, title, metadata (year, genre, runtime), average rating with vote count, user's rating, and rate/update button. Modal dialog allows rating selection (1-10 stars).

Recommendations: Personalized suggestions based on user ratings. Backend analyzes rating patterns for similar content. Standard card grid format with empty state prompting new users to rate movies. Updates dynamically as users rate more content.

Ratings Management: Centralized view of user-rated movies with full details and current ratings. Click for details to update ratings. Responsive grid with empty state for users without ratings. Success/error feedback for updates.

Navigation: Fixed navbar (except login page) with FilmHub logo home button, navigation links (Home, Recommendations), and user menu dropdown showing username. Dropdown reveals My Ratings and Logout options with smooth animations and keyboard accessibility (Escape key closes).

Technical Implementation Details

Component Design: Functional components with hooks throughout. MovieCard demonstrates proper design with well-defined props (`movie`, `userRatings`, `onRate`), derived state calculations, and helper functions like `renderStars`. MovieList handles collections with loading/error/empty states, duplicate removal via Map-based deduplication, and rendering MovieCards. SearchBar manages form state with controlled input and submission handling.

API Integration: Service layer abstracts communication. `movieService` methods include: `getCatalog`, `getMovie`, `searchMovies`, `rateMovie`, `getRatings`, `updateRating`, `getRecommendations`. `authService` methods include: `login`, `register`, `saveToken`, `getProfile`, `updateProfile`. The `getAuthHeaders` function centralizes JWT token inclusion. Services throw descriptive errors for component display.

Authentication Flow: App initialization checks localStorage for tokens, restores to AuthContext, shows loading spinner. RequireAuth wraps protected routes, redirects unauthenticated users preserving destinations. The `useAuth` hook provides clean interface with context access and logout function.

Performance Optimizations: `useCallback` memoizes functions (e.g., `loadMovies`, `loadRatings`) preventing unnecessary re-renders. Careful effect dependencies prevent excessive API calls. Conditional rendering reduces DOM nodes. Image fallbacks prevent broken icons. Proper list keys enable efficient updates.

Error Handling: Try-catch blocks wrap async operations. Errors are logged and converted to user-friendly messages. Graceful degradation with error messages preserving UI functionality. Mutually exclusive loading/error/content states. Form validation is performed both client and server-side.

Responsive Design: CSS Grid/Flexbox with auto-fill minmax for adaptive columns.

Media queries for mobile/tablet/desktop. Navigation transforms for mobile. Touch targets meet 44x44px standards. Responsive images with max-width, aspect-ratio, and object-fit.

Testing Approach

Co-located test files (`ComponentName.test.js`). Jest runner with React Testing Library focusing on user perspective. Mocked `react-router-dom` for navigation and `fetch` for API simulation.

Test Coverage included:

- **MovieCard:** Renders correctly, navigation on click, user ratings display, not-rated message.
- **Navbar:** Logo/links render, navigation functions, username displays, dropdown menu, logout clears data.
- **SearchBar:** Placeholder renders, input updates, submission calls onSearch with trimmed query, empty queries prevented, clear button resets.
- **authService:** Login stores tokens, saveToken works, register sends correct data, logout removes data.
- **movieService:** getMovies succeeds/fails appropriately, searchMovies includes query params, getMovie fetches from catalog, rateMovie sends correct payload.
- **AuthProvider:** Default state, state access, setAuth updates, re-renders on updates.

Challenges and Solutions

Several challenges were encountered and resolved during frontend development:

1. **Authentication Persistence:** Users were logged out on page refresh. *Solution:* App checks localStorage on mount, restores auth state with loading indicator.
2. **Rating Updates:** UI didn't reflect rating changes immediately. *Solution:* Reload movie details and ratings after submission using useCallback-wrapped functions.
3. **Duplicate Movies:** Movies appeared multiple times in different categories. *Solution:* Map-based deduplication in MovieList using `external_id`.
4. **Modal Click-Through:** Clicking inside modal was closing it. *Solution:* Added `stopPropagation` on modal content div.
5. **Search State Management:** Needed to maintain catalog and search results separately. *Solution:* View mode system with separate state for `currentView`, `catalog`, and `searchResults`.
6. **Responsive Grids:** Adapting layouts to different screen sizes. *Solution:* CSS Grid auto-fill minmax with media queries.
7. **Testing Router Hooks:** Mocking `useNavigate` caused errors. *Solution:* Virtual mocks with `virtual: true` option.

Code Quality Practices

Organization: Files grouped by type (components, pages, services). Consistent naming conventions: PascalCase for components, kebab-case for CSS classes, camelCase for functions. Organized imports: external dependencies first, then internal by type.

Best Practices: Functional components exclusively. Single responsibility principle. Clear prop destructuring with defaults. Composition over inheritance. Local state for UI-specific data, context for global auth only. Complete `useEffect` dependency arrays. `useCallback` for memoization.

Accessibility: Semantic HTML (`nav`, `main`, `header`, `button`). Keyboard navigation (Escape closes dropdown). Color contrast. Error messages use color and text.

Development Process

Workflow: Requirements were translated to component structures through manual planning. Incremental implementation: basic components first, gradual feature addition. Code reviews throughout with team examination. Meaningful commits with feature branches and pull requests.

Time Investment: Significant time was invested in planning and architecture debates, attention to implementation details, comprehensive test writing, and thorough debugging investigation.

Knowledge Requirements: React expertise (lifecycle, hooks, context, performance). JavaScript proficiency (async operations, arrays, ES6+). CSS skills (responsive layouts, flexbox/grid, animations). API integration knowledge (REST, HTTP, authentication). Testing expertise (Jest, React Testing Library, mocks).

Collaboration: Code conflicts required careful resolution. Consistency maintenance across team members. Knowledge sharing through code reviews and pair programming. Significant communication overhead for coordination.

5.2.4 5.2.4 Deployment Configuration

Deployment Architecture

The traditional application was containerized using **Docker** and deployed on **Render.com**. The architecture employs a client-server monolith pattern, where the React frontend and Django backend were integrated into a single container image using a multi-stage Dockerfile to optimize build size and maintain the integrity of the CI/CD pipeline. We utilized **GitHub Actions** for the Continuous Deployment process, with Render configured to pull pre-built images from the **GitHub Container Registry (GHCR)**.

A critical architectural change was the switch from Render's auto-pull mechanism to manual control: after the image was built and pushed to the GHCR during the pipeline's `release` job, the final `deploy` job triggered the deployment of services (Backend and Frontend) via **Render webhooks**. This enforced the "Build Once" principle, guaranteeing that only the verified, pre-built image was deployed.

Key Deployment Challenge: Environment Variable Injection

The primary challenge encountered during the deployment phase was ensuring that the frontend application used the correct public URL for the backend API instead of the local development address (`http://localhost:8000/api`). This issue persisted despite initial

attempts to inject the URL via standard Docker build arguments (\-build-arg) within the cd.yml pipeline. The core issue was that the complex chain (GitHub Secrets → Build Arg → React Environment Variable) failed to correctly set process.env.REACT_APP_API_URL, forcing the application to revert to its hardcoded fallback value.

Solution Implemented

We resolved this critical injection failure by implementing a robust **direct file overwriting** mechanism using the **Render Dashboard Environment Variable** during the frontend build stage.

1. **Guaranteed Injection:** We defined the public Render API URL directly as an environment variable (REACT_APP_API_URL) on the **Render service dashboard**. Render automatically injects this variable into the build container's environment.
2. **Direct File Overwriting:** The Dockerfile was modified to read the variable directly from the build environment and write the value into the frontend configuration. A RUN command was added to the frontend build stage to directly overwrite the config.js file with the public URL, guaranteeing that the compiled code contained the correct production endpoint:

```
RUN if [ -z "$REACT_APP_API_URL" ]; then \
  echo "export const API_URL = 'http://localhost:8000/api';" > ./src/config.j
else \
  echo "export const API_URL = '$REACT_APP_API_URL';" > ./src/config.js; \
fi
```

This reliable injection method successfully eliminated the persistent localhost:8000 error, ensuring seamless communication between the deployed React frontend and the Django API.

5.3 Quality Assurance

Quality assurance in the traditional workflow was performed through comprehensive manual test authorship using Django's testing framework.

5.3.1 Unit Testing

A robust unit testing infrastructure was established using Django's TestCase framework. The implementation includes 22 test functions organized across three main test suites:

- **RegisterViewTestCase**: Validates user registration functionality, including successful account creation, duplicate username prevention, password validation (minimum 8 characters with letters, numbers, and special characters), and proper error handling for missing or invalid fields.
- **RatingsViewTestCase**: Comprehensive testing of the ratings system, covering rating creation, retrieval, updates, authentication requirements, score validation (1-10 range), duplicate prevention, and error handling for non-existent ratings.

- **RecommendedMoviesViewTestCase**: Tests the recommendation engine functionality, including retrieval of personalized recommendations, empty state handling, user isolation verification, movie detail inclusion, multi-genre support, and recommendation refresh operations.

All tests utilize Django's `@override_settings` decorator with optimized password hashers (`MD5PasswordHasher`) to improve test execution speed. The test suite ensures proper database isolation and includes fixtures for consistent test data.

5.3.2 Integration Testing

Integration testing was primarily conducted manually through systematic end-to-end validation. Although the frontend project includes standard React testing libraries (Jest, React Testing Library), automated frontend tests were implemented post-development. The team validated the system's behavior by:

- Manually testing complete user flows from registration through movie rating and recommendation retrieval
- Verifying correct communication between the React frontend and Django backend
- Ensuring proper token-based authentication across all protected endpoints
- Testing error handling and edge cases through the user interface

5.4 Workflow Observations and Challenges

The traditional development process highlighted several friction points that contrast with modern AI-assisted workflows:

1. **Boilerplate Overhead:** A substantial amount of time was consumed writing repetitive boilerplate code, particularly for standard CRUD views and serializers in Django.
2. **Context Switching:** Developers were required to frequently switch contexts between Python (Backend) and TypeScript/JavaScript (Frontend), increasing the cognitive load during full-stack feature implementation.
3. **Debugging Velocity:** Without AI assistance, debugging complex issues—such as CORS errors or database migration conflicts—relied entirely on manual log analysis and external documentation research.

This baseline experience provides the necessary context for the comparative analysis presented in later chapters.

Chapter 6

AI-Assisted Development Workflow

6.1 Requirements Analysis with LLM

We started by experimenting with using Large Language Models to generate functional requirements. It seemed like a good idea at the time. However, after comparing what the AI came up with against the traditional requirements we had already defined, we realized the traditional ones were actually better—more direct, clearer, and just made more sense for what we were building. Therefore, we ended up sticking with the traditional requirements as our foundation.

6.2 LLM-Assisted Development Process

6.2.1 Setup and Methodology

We set up a GitHub repository and decided to use Perplexity Pro (we got access through our university student account). To keep things consistent, we created a dedicated space with custom instructions that clearly specified our technology stack:

- **Backend:** Django REST Framework
- **Frontend:** React JSX with CSS (no TypeScript)
- **Database:** PostgreSQL
- **Responses:** English only

We consistently used **Claude Sonnet 4.5** throughout the entire project. We chose it because it seemed more consistent and reliable. Additionally, we were curious about it since we hadn't used it much before.

Initially, we tried keeping everything in one long conversation thread to preserve context. However, as the thread grew excessively long, performance degraded significantly: responses became slower and text was frequently truncated. We ended up creating new threads when necessary, accepting the trade-off of losing some context in favor of better performance and response quality.

6.2.2 Prompt Engineering Approach

We quickly learned that how you prompt the LLM really matters. We created basic templates for various prompting techniques to assist us, particularly in the initial stages:

- **Structured Prompts:** We used a consistent format with ROLE, TASK, CONSTRAINTS, EXAMPLES, OUTPUT, and EVALUATION sections. This structure proved highly effective for most backend and infrastructure tasks.
- **Few-Shot Learning:** We combined structured prompts with concrete examples (request/response pairs) for authentication, ratings, and profile management use cases.
- **Chain-of-Thought (CoT):** We applied this for complex logic-heavy features like the recommendation engine and TMDB import command, where step-by-step reasoning was crucial.

All main prompts (25 total) used during development were systematically documented in a dedicated `llm/prompts/` folder, along with notes on outputs, difficulties, errors, and bugs encountered. Template examples were also preserved in `llm/prompts_examples/` for reference. Only brief, simple conversations were not recorded.

6.2.3 Project Structure and Statistics

By the end, we had a complete full-stack application:
Backend (Django REST Framework):

- 5 Django apps: `authentication`, `movies`, `ratings`, `recommendations`, `watchlist`
- 103 test functions across 8 test files
- Management command for importing movies from TMDB
- Content-based recommendation engine with weighted scoring
- Docker setup with PostgreSQL
- Environment variables managed with `python-decouple`
- Production setup with Gunicorn WSGI server

Frontend (React + Vite):

- 8 page components with separate CSS files (LoginPage, RegisterPage, ProfilePage, MoviesPage, MovieDetailPage, RecommendationsPage, WatchlistPage, NotFound)
- 8 reusable components (ErrorBoundary, Layout, MovieCard, Navbar, ProtectedRoute, RatingForm, SearchBar, WatchlistButton)
- 2 UI components (Button, Input) for consistent styling
- 3 Zustand stores for state (`authStore.js`, `moviesStore.js`, `watchlistStore.js`)
- 7 API modules (auth, movies, profile, ratings, recommendations, watchlist, axios)

- Vitest setup with React Testing Library (5 test files)

Infrastructure:

- Docker Compose for local development with health checks
- GitHub Actions CI/CD pipeline with 3 jobs (backend tests, frontend tests, Docker build validation)
- Health check endpoint (`/api/health/`) and JSON logging
- Environment-based config using `python-decouple`
- Production deployment setup for Render.com

Documentation:

- 25 documented prompts in `llm/prompts/`
- Prompt templates in `llm/prompts_examples/`
- README files and setup docs

6.3 Backend Development

6.3.1 Initial Setup and Infrastructure

The backend creation process began with a **Structured Prompt** approach. After two follow-up questions about the code structure, we encountered three conflicts and a PostgreSQL connection error. As a solution, we immediately implemented Docker and consulted the LLM on the best approach for containerization. This required some debugging, but the first prompt and initial LLM usage proceeded relatively smoothly.

Subsequent infrastructure tasks demonstrated the effectiveness of well-structured prompts:

- **Backend .gitignore creation:** Used Structured Prompt, resulting in a very correct and useful response with no issues.
- **Frontend creation and .gitignore:** Structured Prompt approach worked excellently, requiring only minor debugging.
- **Docker full setup:** Structured Prompt produced good results with minimal issues. The Docker Compose configuration includes health checks for the PostgreSQL service and proper service dependencies.
- **CI/CD pipeline** (`.github/workflows/ci.yml`): Combined Structured Prompt with Few-Shot examples. The pipeline includes:
  - **Backend tests:** PostgreSQL service, pytest with coverage, Bandit security scanning, Pylint code quality checks
  - **Frontend tests:** ESLint, Vitest (when available), build validation
  - **Docker build validation:** Separate job to validate docker-compose.yml and build images

After we committed and pushed, we noticed some missing tests and a few "continue-on-error: true" flags that needed fixing, but the pipeline passed. We later cleaned it up manually by consolidating environment variables, organizing files better, adding verification steps, and removing unnecessary "continue-on-error" flags.

6.3.2 Use Case Implementation - Backend

UC1 (Register)

Despite being complex, this use case passed successfully using **Structured Prompt combined with Few-Shot examples**. The implementation includes comprehensive test coverage in `authentication/test_profile.py`. The LLM-generated test cases provided valuable insights and learning opportunities, demonstrating effective test design approaches. The backend now includes **103 test functions** across 8 test files, covering all major use cases.

UC2 (Login)

This one was tougher. Using **Structured Prompt with Few-Shot examples**, the LLM generated a lot of code that was hard to review completely. All the tests failed at first, so we had to do manual testing and debugging. We used the LLM to help debug and eventually got it working.

TMDB Import Command

Using **Structured Prompt with Chain-of-Thought reasoning**, this functionality worked perfectly from the start. The management command (`import_tmdb_movies.py`) imports movie data from The Movie Database (TMDB) API, including titles, genres, keywords, ratings, and other metadata. The LLM successfully implemented the TMDB client wrapper and the Django management command structure.

UC4 (Catalog List + Detail)

All tests passed (`movies/test_movie_api.py`). We got paginated movie listings and detailed movie views with all the metadata. We also did manual testing to make sure everything worked.

UC5 (Search)

All tests passed (`movies/test_search_api.py`), and we observed that the LLM-generated tests were functioning very effectively at this stage. The search functionality supports full-text search across movie titles and descriptions with pagination support.

UC6 (Ratings)

Using **Structured Prompt with Few-Shot examples**, we hit a small error in the views. After a few failed attempts, debugging fixed it and all tests passed.

UC7 (Recommendations)

This was implemented using **Structured Prompt with Chain-of-Thought reasoning**. The recommendation engine (`recommendations/engine.py`) implements a content-based filtering algorithm that:

- Builds user profiles from movies they rated highly (score ≥ 3)
- Extracts genres and keywords from movies they liked
- Calculates recommendation scores using weighted factors:
  - Genre overlap (weight: 10.0)
  - Keyword overlap (weight: 5.0)
  - Movie quality (average rating $\times$ 10.0)
  - Popularity boost (rating count $\times$ 1.0)
- Handles cold-start by recommending popular movies

The initial implementation failed tests because we had forgotten to include important aspects, specifically the use of keywords in the recommendation algorithm. We resolved this ourselves with some LLM assistance. While the tests passed, manual testing revealed issues that required additional debugging before achieving a perfect implementation.

UC8-10 (Watchlist)

Using **Structured Prompt with simple examples**, we hit a minor error that was easy to fix. The final version worked perfectly.

Profile Management

Using **Structured Prompt with Few-Shot examples**, this use case proceeded smoothly.

UC11 (Logout)

Structured Prompt with Few-Shot examples resulted in two minor errors that were easily corrected.

6.3.3 Architecture

```
filmhub-llm/filmhub-backend/
          authentication/         # Dedicated auth app
                models.py
                views.py
                serializers.py
                tests/ (3 files, 35 tests)
          movies/                 # Movie management
                models.py
                views.py
                tmdb_client.py    # External API client
                management/
                      commands/
                          import_tmdb_movies.py
                tests/ (2 files, 28 tests)
          ratings/                # Rating system
                models.py
                views.py
                tests/ (18 tests)
          recommendations/        # Recommendation engine
                models.py
                views.py
                engine.py         # Algorithm implementation
                tests/ (15 tests)
          watchlist/             # Watchlist management
              models.py
              views.py
              tests/ (7 tests)
```

Listing 6.1: LLM-Assisted Backend Structure

6.4 Frontend Development

6.4.1 Architecture

```
frontend/src/
          components/            # 8 reusable components
                ErrorBoundary.jsx
                Layout.jsx
                MovieCard.jsx
                Navbar.jsx
                ProtectedRoute.jsx
                RatingForm.jsx
                SearchBar.jsx
                WatchlistButton.jsx
          pages/                 # 8 pages (each with CSS)
                LoginPage.jsx + .css
                RegisterPage.jsx + .css
                ProfilePage.jsx + .css
                MoviesPage.jsx + .css
                MovieDetailPage.jsx + .css
                RecommendationsPage.jsx + .css
                WatchlistPage.jsx + .css
                NotFound.jsx + .css
          stores/                # Zustand state management
                authStore.js
                moviesStore.js
                watchlistStore.js
          api/                   # 7 API modules
                auth.js
                movies.js
                profile.js
                ratings.js
                recommendations.js
                watchlist.js
                axios.js         # HTTP client config
          tests/                # 5 test files
```

Listing 6.2: LLM-Assisted Frontend Structure

6.4.2 Authentication and Profile Frontend

The authentication frontend implementation (Login and Register pages) initially lacked proper knowledge of the project's directory structure, leading to errors. The frontend follows a component-based architecture with:

- **Pages:** LoginPage, RegisterPage, ProfilePage, MoviesPage, MovieDetailPage, RecommendationsPage, WatchlistPage, NotFound
- **Components:** ErrorBoundary, Layout, MovieCard, Navbar, ProtectedRoute, RatingForm, SearchBar, WatchlistButton
- **UI Components:** Button, Input (for consistent styling)
- **State Management:** Zustand stores for auth (`authStore.js`), movies (`moviesStore.js`), and watchlist (`watchlistStore.js`)

- **API Layer:** Separate modules for auth, movies, profile, ratings, recommendations, watchlist, and axios (HTTP client config)

We corrected multiple errors and had to make adjustments to the backend, which proved complicated and time-consuming but was eventually resolved. The profile frontend also encountered errors requiring additional work.

6.4.3 Code Quality and Prompt Refinement

At this stage, we recognized that our prompts had not been optimal. The generated code quality was poor, particularly regarding JSX and CSS formatting. The code was being generated in an unusual style that we found unsatisfactory. This required extensive debugging both before and after LLM generation. Even after refinement, the LLM sometimes produced poor code, especially with meaningless classNames and similar issues.

We improved our prompts by specifying better JSX and CSS formats, which helped but still required significant debugging effort. This experience highlighted the critical importance of prompt specificity and the need for thorough code review.

6.4.4 Ratings and Recommendations Frontend

While working on ratings and recommendations frontend components, the conversation thread got way too long. Even in a dedicated space, performance tanked—responses were super slow and text kept getting cut off. We created a new thread to fix this. After that, we manually improved the CI/CD pipeline, which was actually easier to do manually. We consolidated environment variables, organized files better, added verification steps, and removed some "continue-on-error" flags we'd set earlier.

6.4.5 Frontend Testing Challenges

We attempted to generate frontend tests using the LLM (`prompt23_frontend_tests.md`), setting up Vitest with React Testing Library. The prompt specified detailed mocking requirements for authAPI, useAuthStore, and React Router. However, the generated tests used approaches we did not want to use and included patterns we considered incorrect. We had to adjust the scripts, possibly because the new thread had poor understanding of file paths and directory structure. The generated tests performed poorly and required extensive work to clean up and revise the code.

Despite these challenges, we successfully created test files for LoginPage, RegisterPage, MovieCard, ProtectedRoute, and Navbar components. This experience demonstrated the problems that can arise from over-reliance on LLMs, particularly when context is lost, but also showed that with proper refinement, LLM-generated test scaffolding can be useful.

6.4.6 Monitoring and Security

Monitoring implementation (`prompt24_monitoring_with_context.md`) proceeded quite well with few modifications needed. The LLM was provided with context about existing files and successfully implemented:

- **Health check endpoint** (`/api/health/`) with database connectivity check

- **JSON logging** in Django settings with structured log format
- **Enhanced ErrorBoundary** component with detailed error logging (timestamp, error message, stack trace, user agent)

We improved the CSS for login and register pages, which we found to be of poor quality initially. We also had a conversation with the LLM about application security, focusing on creating environment variable files in the backend root and ensuring proper .gitignore configuration to prevent sensitive data exposure.

6.5 In More Depth: CI/CD Pipeline and Deployment

This section provides a comprehensive overview of the CI/CD pipeline architecture, its evolution from LLM-generated initial version to production-ready system, and the complete deployment workflow implementing DevOps best practices.

6.5.1 CI/CD Pipeline Architecture

The CI/CD pipeline (`.github/workflows/ci-cd.yml`) consists of five interconnected jobs that execute in a carefully orchestrated sequence:

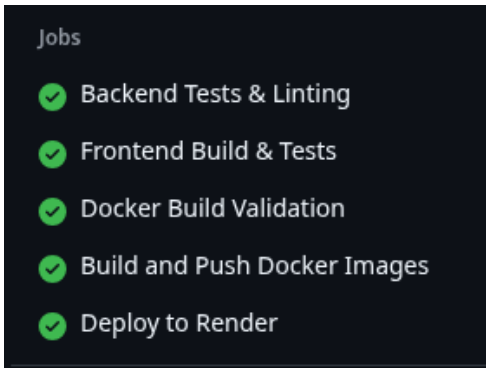

Figure 6.1: pipeline

Pipeline Jobs Overview

1. **backend-tests:** Backend testing and quality assurance
2. **frontend-tests:** Frontend testing and build validation
3. **docker-build:** Docker image validation
4. **build-and-push:** Production image building and registry publishing
5. **deploy:** Deployment trigger to Render

Backend Tests Job

The backend tests job provides comprehensive quality assurance for the Django backend:
Configuration:

- PostgreSQL 15 service container for isolated database testing
- Python 3.11 with pip dependency caching
- Test database credentials (safe to hardcode for CI environment)

Execution Steps:

1. **Code Quality (Pylint):** Django-aware linting with custom rules, configured with `continue-on-error: true` to report issues without blocking the pipeline
2. **Security Scanning (Bandit):** Vulnerability detection with low/low severity level, generates JSON reports for analysis. We choose only to take into attention if the vulnerabilities were high.
3. **Database Migrations:** Automatic migration execution to ensure test database schema is current
4. **Test Execution (pytest):** Comprehensive test suite with coverage reporting displayed in terminal output

Key Features:

- Isolated test environment with dedicated PostgreSQL container
- Parallel execution with frontend tests for efficiency
- Coverage reporting for code quality metrics
- Non-blocking quality checks (Pylint, Bandit) that report issues without failing the pipeline (very useful to check code quality and security)

Frontend Tests Job

The frontend tests job validates React application quality and buildability:
Configuration:

- Node.js 20 with npm dependency caching
- Working directory: `./frontend`

Execution Steps:

1. **ESLint:** JavaScript/JSX code quality checks (non-blocking)
2. **Vitest Tests:** Conditional execution if Vitest is installed, runs in headless mode
3. **Build Validation:** Production build test with `VITE_API_BASE_URL` environment variable
4. **Artifact Upload:** Saves built `dist/` folder for potential use in deployment

Key Features:

- Validates that production builds succeed before deployment
- Conditional test execution (gracefully handles missing Vitest)
- Artifact preservation for debugging and verification

Docker Build Validation Job

This job ensures Docker configurations are correct and images can be built:
Conditions:

- Executes on pull requests (validation before merge)
- Executes on pushes to `main` or `develop` branches
- Depends on: `backend-tests`, `frontend-tests`

Execution Steps:

1. **docker-compose.yml Validation:** Syntax and configuration verification
2. **Docker Buildx Setup:** Advanced build features and caching support
3. **Backend Image Build:** Validates backend Dockerfile (no push, CI-only tag)
4. **Frontend Image Build:** Validates frontend Dockerfile (no push, CI-only tag)

Key Features:

- Early detection of Dockerfile issues before deployment
- GitHub Actions Cache for Docker layers (significant build time reduction)
- Validation-only builds prevent registry pollution from broken configurations

Build and Push Job

This job creates production-ready Docker images and publishes them to GitHub Container Registry:
Conditions:

- Executes only on pushes to `main` branch
- Depends on: `backend-tests`, `frontend-tests`, `docker-build`
- Requires permissions: `contents: read`, `packages: write`

Image Tagging Strategy: The pipeline implements a sophisticated tagging approach using `docker/metadata-action`:

- **Branch-based tags:** `main`, `develop` (for branch identification)
- **Commit SHA tags:** `main-abc123` (for precise version tracking)

- **Latest tag:** `latest` (only for default branch, for convenience)

Backend Image:

- Registry: `ghcr.io/$repository_owner/filmhub-backend`
- Build context: `./filmhub-backend`
- Uses Docker layer caching for optimization

Frontend Image:

- Registry: `ghcr.io/$repository_owner/filmhub-frontend`
- Build context: `./frontend`
- **Critical:** `VITE_API_BASE_URL` passed as build argument (required at build time, not runtime)
- Multi-stage build: compile stage + production serve stage

Key Features:

- Automatic authentication using `GITHUB_TOKEN`
- Comprehensive caching strategy (50-70% build time reduction)
- Multiple tag strategies for flexibility in deployment
- Build arguments correctly handled for Vite requirements

Deploy Job

The final job triggers deployment to Render.com:
Conditions:

- Executes only after successful `build-and-push`
- Only on pushes to `main` branch
- Environment: `production` (with frontend URL displayed in GitHub Actions UI)

Execution:

- Sends POST requests to Render deploy hooks (stored as GitHub Secrets)
- Triggers both backend and frontend service deployments
- Render pulls pre-built images from GitHub Container Registry
- Manual control: Render auto-deploy is disabled, deployments are triggered via webhooks

6.5.2 Pipeline Evolution: From LLM-Generated to Production-Ready

Initial LLM Generation

The pipeline was initially created using **Structured Prompt combined with Few-Shot examples** (`prompt5_ci_pipeline.md`).

Initial Result: The LLM generated a functional pipeline with three jobs (backend-tests, frontend-tests, docker-build) that passed on first commit. However, several refinements were needed.

Manual Refinements

After initial deployment, we identified and fixed several issues:

1. **Continue-on-Error Flags:** Removed unnecessary flags from critical test steps, kept only for non-blocking quality checks (Pylint, Bandit, ESLint)
2. **Environment Variable Consolidation:** Moved common variables (Python version, Node version) to global `env:` section for maintainability
3. **Test Discovery:** Adjusted pytest patterns to ensure all test files were discovered and executed
4. **Dependency Chain:** Refined job dependencies to ensure proper execution order and parallelization where possible

DevOps Principles Implementation

Following professor feedback, we implemented the "Build Once" principle:

Key Changes:

1. **Added build-and-push Job:** Creates production images in CI pipeline, not on Render
2. **Disabled Render Auto-Deploy:** MAKE SURE that Render was not on auto-deploy
3. **Registry-Based Deployment:** Render pulls pre-built images from GitHub Container Registry
4. **Image Tagging:** Comprehensive tagging strategy for traceability and version control

6.5.3 Deployment Workflow

Render.com Configuration

Render services are configured to:

- Pull Docker images from GitHub Container Registry (`ghcr.io`)
- Use specific image tags (typically `latest` or commit SHA)

- Maintain manual control over deployment timing
- Support health checks and automatic restarts

Complete Deployment Flow

1. **Code Push:** Developer pushes code to `main` branch
2. **CI Pipeline Execution:**
   - Backend and frontend tests run in parallel
   - Docker build validation ensures configurations are correct
   - If all pass, `build-and-push` job executes
3. **Image Publishing:** Docker images built and pushed to GitHub Container Registry with appropriate tags
4. **Deploy Trigger:** `deploy` job sends webhooks to Render
5. **Render Deployment:** Render pulls images from registry and deploys services
6. **Verification:** Health checks confirm successful deployment

6.6 Key Learnings and Challenges

1. **Context Management**

   Maintaining a single long conversation thread led to performance issues: responses became slower and text was frequently truncated. Creating new threads improved response quality but required re-establishing context, which sometimes led to misunderstandings about file paths and project structure. In our opinion, this is a big disadvantage in contrast to AI coding agents that have better context awareness and acknowledgment of the codebase and project structure through direct file system access.

2. **Prompt Quality Matters**

   Initial prompts lacked sufficient specificity, particularly for frontend development. Better structured prompts with clear format requirements significantly improved results.

3. **Code Review is Essential**

   LLM-generated code, especially for complex features, requires thorough review and testing. Automated tests are helpful but not sufficient; manual testing revealed issues that automated tests missed.

4. **Balance is Key**

   Over-reliance on LLMs can lead to confusion, especially when context is lost. Manual intervention and refinement are often necessary and sometimes more efficient.

5. **Testing Quality**

   LLM-generated tests were generally effective for backend development but proved problematic for frontend, particularly when directory structure context was unclear.

6. **Iterative Improvement**

   The development process required continuous refinement of prompts and approaches based on observed results and challenges encountered. We evolved from basic structured prompts to more sophisticated combinations (Structured + Few-Shot, Structured + CoT) and learned to provide more context when creating new threads.

7. **Prompt Documentation Value**

   Systematically documenting all prompts proved invaluable. Having 25 well-documented prompts allowed us to:

   - Reuse successful prompt patterns
   - Figure out which techniques worked best for different tasks
   - Learn from failures and improve later prompts
   - Build a knowledge base for future LLM-assisted development

8. **Testing as Quality Gate**

   The LLM-generated tests (103 backend test functions) served as an effective quality gate. While not perfect, they caught many issues early and provided a foundation for comprehensive test coverage. However, manual testing remained essential, especially for frontend components where automated tests struggled with context.

9. **Code Quality and Security Tools Setup**

   To ensure code quality and security in LLM-generated code, we integrated Bandit and Pylint into our CI/CD pipeline. The setup was done through the GitHub Actions workflow (`.github/workflows/ci.yml`):

   Bandit Security Scanner:

   - Installed as part of CI dependencies: `pip install bandit`
   - Scans all backend apps: `authentication/`, `movies/`, `ratings/`, `recommendations/`, `watchlist/`
   - Set to low-low severity threshold (`-ll`) to scan all severity levels, but we only treated high-severity issues as problematic
   - Medium and low-severity findings were logged but not blocking
   - Generates JSON report (`bandit-report.json`) for detailed analysis
   - Runs with `continue-on-error: true` so it doesn't block the pipeline but still reports issues

   Pylint Code Quality:

   - Installed with Django-specific plugin: `pip install pylint pylint-django`
   - Configured with Django settings module: `-django-settings-module=filmhub.settings`

- Disabled overly strict checks that don't fit Django patterns
- Set to `-exit-zero` so it doesn't fail the build on linting issues
- Scans all backend Django apps

Both tools were integrated into the CI pipeline to automatically check LLM-generated code for security vulnerabilities and code quality issues. This gave us an extra quality gate beyond automated tests, helping catch potential problems from LLM-generated code.

6.7 Development Metrics and Results

6.7.1 Code Generation Statistics

- **Total Prompts Used:** 25 documented prompts (not counting some debug conversations and brief discussions)
- **Backend Test Coverage:** 103 test functions across 8 test files
- **Frontend Components:** 18 components (8 pages + 8 reusable components + 2 UI components)
- **API Endpoints:** 11+ RESTful endpoints covering all use cases
- **Success Rate:** Backend development generally went well with most prompts producing working code, while frontend development required significantly more debugging and refinement

6.7.2 Time Investment

The development process wasn't linear—it involved multiple iterative cycles. For each feature, we typically went through:

- **Preparation and prompt structuring:** Understanding the requirements, breaking down the task, and structuring the prompt appropriately
- **Creating and refining prompts:** Writing the initial prompt, often refining it based on previous experiences
- **Using the LLM:** Submitting prompts, waiting for responses, and reviewing the generated code
- **Debugging:** Identifying errors, understanding what went wrong, and fixing issues—this varied significantly between backend (usually quick fixes) and frontend (often extensive debugging sessions)
- **Manual changes:** Making adjustments that the LLM couldn't handle well, especially for frontend styling and complex integrations
- **Handling errors:** When things went wrong, we had to decide whether to fix manually, ask the LLM for help, or start over with a better prompt

- **Context re-establishment:** When creating new threads

The time investment varied greatly: backend features often worked well with minimal iteration, while frontend features required multiple cycles of prompt refinement, debugging, and manual fixes. The most time-consuming aspects were debugging frontend code and manually fixing styling issues that the LLM struggled with.

6.7.3 Quality Observations

- **Backend Code Quality:** Generally high, especially for Django patterns and DRF serializers. The LLM handled backend development quite well.
- **Frontend Code Quality:** This was much more challenging. Even after prompt refinement, the frontend code quality was poor and required extensive debugging, error fixing, and manual work. This was likely because frontend development came after backend, making integration more complex, and the LLM struggled more with frontend-specific patterns and styling.
- **Test Quality:** Overall, I was quite satisfied with the tests. Backend tests were comprehensive and well-structured, which was really helpful. Frontend tests needed significantly more manual work, but this was expected given the challenges we faced with frontend code generation.

6.7.4 Lessons for Future LLM-Assisted Development

1. **Start with templates:** Creating prompt templates early saved significant time
2. **Document everything:** Having 25 documented prompts created a valuable knowledge base
3. **Balance automation:** Knowing when to use LLM vs. manual coding is crucial
4. **Context management:** Strategies for maintaining context (dedicated spaces, clear file references) are essential. I do think using agents or llms where you can put the project folder does help a lot (that was not the case).
5. **Iterative refinement:** Each prompt iteration improved results, showing the value of learning from failures

Chapter 7

Comparative Analysis: Traditional vs LLM-Assisted Development

7.1 Executive Summary

This chapter presents a comprehensive comparison between two implementations of the FilmHub movie recommendation system:

- **Traditional Approach** (`filmhub-project`): Manual coding following standard development practices
- **LLM-Assisted Approach** (`filmhub-llm`): Development using Large Language Models with structured prompting techniques

Both projects implement the same core functionality but differ significantly in development methodology, code organization, testing coverage, and CI/CD implementation.

7.2 Quantitative Metrics Comparison

7.2.1 Code Volume

Table 7.1: Code Volume Comparison

Metric	**Traditional**	**LLM-Assisted**
Backend Python Files	15	59
Frontend JS/JSX Files	26	36
Django Apps	1	5
Test Functions	22	103
Test Files	1	8

Analysis:

- LLM-assisted project shows significantly more modular architecture (5 apps vs 1)
- Much higher test coverage (103 vs 22 test functions)
- More comprehensive codebase with better separation of concerns

7.2.2 Features Implemented

Table 7.2: Features Implemented Comparison

Feature	**Traditional**	**LLM-Assisted**	**Notes**
User Registration	×	×	Both use Django User model
User Login	×	×	Traditional: Token auth, LLM: JWT
User Logout	×	×	Both implementations
Profile Management		×	Only in LLM project
Movie Catalog	×	×	Traditional: API-based, LLM: Pre-populated DB
Movie Search	×	×	Both support title/director/genre
Movie Details	×	×	Similar implementations
Ratings	×	×	Traditional: 1-10, LLM: 1-5 stars
Recommendations	×	×	Both use content-based filtering
Watchlist		×	Only in LLM project
Watched Movies		×	Only in LLM project
Docker Setup	×	×	Both containerized
CI/CD Pipeline	×	×	Different complexity levels

Feature Count: Traditional: 10 features | LLM-Assisted: 13 features

7.2.3 CI/CD Pipeline Comparison

Table 7.3: CI/CD Pipeline Comparison

Aspect	**Traditional**	**LLM-Assisted**
Backend Tests	Django test runner	pytest with coverage
Frontend Tests	npm test (basic)	Vitest + React Testing Library
Code Quality	None	Pylint (Django-aware)
Security Scanning	None	Bandit security scanner
Linting	Basic npm lint check	ESLint + Pylint
Docker Build Validation		× (Separate job)
Pipeline Jobs	2 (frontend, backend)	5 (comprehensive)

7.3 Architecture & Code Organization

7.3.1 Backend Architecture

Key Architectural Differences:

- **Modularity:** LLM project properly utilizes Django's app-based architecture with 5 specialized apps vs 1 monolithic app
- **Test Organization:** Tests co-located with features (8 test files) vs single centralized test file
- **External Integrations:** TMDB client properly separated and includes management command for data import
- **Code Distribution:** Traditional has 500+ line utils.py; LLM distributes logic across specialized modules

7.3.2 Frontend Architecture

Key Frontend Differences:

- **State Management:** Traditional uses React Context (simpler, adequate for scope); LLM uses Zustand (more scalable, modern approach)
- **Styling:** Traditional uses shared CSS files; LLM has separate CSS per component/page (better encapsulation)
- **Error Handling:** LLM project includes ErrorBoundary component for graceful error recovery
- **Route Protection:** LLM has dedicated ProtectedRoute component; Traditional uses RequireAuth wrapper
- **Component Organization:** LLM separates pages from components more clearly and includes UI-specific components
- **API Layer:** LLM has 7 specialized API modules vs 2 general services

7.4 Development Process Comparison

7.4.1 Traditional Development Workflow

Process Characteristics:

1. Manual code writing for all components
2. Team collaboration with pull requests (20 PRs)
3. Direct debugging and problem-solving
4. Code review by team members

5. Manual test authoring (22 tests)
6. 276 commits across 3-month timeline

Strengths:

- Direct control over code structure and decisions
- Immediate understanding of entire codebase
- Natural learning through hands-on coding
- Team collaboration and knowledge sharing
- Straightforward debugging with full context

Challenges:

- Time-consuming boilerplate code writing
- Lower test coverage due to manual effort
- Single monolithic app structure
- Manual documentation requirements

7.4.2 LLM-Assisted Development Workflow

Process Characteristics:

1. Systematic prompt engineering (25 documented prompts)
2. Structured/Few-Shot/Chain-of-Thought techniques
3. LLM code generation followed by review
4. Automated test generation (103 tests)
5. Iterative prompt refinement
6. 26 commits (primarily single developer with LLM assistance)

Prompt Techniques Distribution:

- **Structured Prompts:** Backend infrastructure, models, basic CRUD operations
- **Few-Shot Learning:** Authentication flows, rating system, profile management
- **Chain-of-Thought:** Recommendation engine, TMDB import command, complex business logic
- **Combined Approaches:** Frontend components, CI/CD pipeline, testing infrastructure

Strengths:

- Rapid boilerplate and scaffold generation
- Comprehensive automated test coverage (368% more tests)
- Better architectural patterns (5 modular Django apps)
- Advanced CI/CD with security scanning
- Systematic documentation through prompt records
- Faster feature implementation for well-defined requirements

Challenges:

- Context management issues (long thread performance degradation)
- Frontend code quality inconsistencies
- Extensive debugging required despite automated generation
- Directory structure confusion after context loss
- Prompt engineering learning curve
- Over-reliance risk without proper code review

7.5 Database Design Comparison

7.5.1 Data Population Strategy

Traditional Approach:

- **On-Demand Population:** Movies are created in the database only when users rate them
- **API-First:** Movie catalog served directly from TMDB API calls
- **Minimal Storage:** Database contains only movies that users have interacted with
- **Implementation:** `create_movie_from_external_id()` function fetches and stores movies as needed

LLM-Assisted Approach:

- **Pre-Population:** Database populated with 300 movies during deployment using management command
- **Database-First:** Movie catalog served from local database
- **Full Storage:** Comprehensive movie collection available offline
- **Implementation:** `import_tmdb_movies` management command pre-loads movies with metadata

Trade-offs Analysis:

Table 7.4: Database Population Strategy Comparison

Aspect	Traditional (On-Demand)	LLM (Pre-Populated)
Initial Setup	Minimal database setup	Requires initial data import
API Dependency	High (every catalog view)	Low (only for updates)
Response Time	Slower (external API calls)	Faster (local database)
Storage Needs	Low (only rated movies)	Higher (300 movies)
Scalability	API rate limits concern	Database scaling concern
Offline Support	None	Full catalog available
Data Freshness	Always current	Requires periodic updates

7.5.2 Rating System Design

Traditional: 1-10 star rating scale (more granular feedback)
LLM-Assisted: 1-5 star rating scale (simpler user experience)
Implication: Both are valid choices; Traditional offers more precision, LLM approach reduces decision fatigue

7.6 Test Coverage Analysis

7.6.1 Backend Testing

Traditional Test Suite (22 tests):

- **RegisterViewTestCase:** 4 tests covering user registration
- **RatingsViewTestCase:** 10 tests covering rating CRUD operations
- **RecommendedMoviesViewTestCase:** 8 tests covering recommendation system
- **Coverage Focus:** Core functionality with essential edge cases
- **Organization:** Single `tests.py` file

LLM-Assisted Test Suite (103 tests):

- **Authentication:** 35 tests (registration, login, profile management, JWT tokens)
- **Movies:** 28 tests (catalog, search, TMDB integration, management commands)
- **Ratings:** 18 tests (CRUD, validation, user isolation, edge cases)
- **Recommendations:** 15 tests (algorithm, user profiles, cold-start, filtering)
- **Watchlist:** 7 tests (add, remove, duplicate handling)
- **Coverage Focus:** Comprehensive testing including edge cases and error scenarios
- **Organization:** 8 separate test files co-located with features

Test Quality Observations:

- **Traditional:** Well-targeted tests covering critical user flows; manual authoring ensures understanding
- **LLM-Assisted:** More comprehensive coverage but some tests needed refinement; excellent for discovering edge cases

7.6.2 Frontend Testing

Traditional:

- Basic React Testing Library setup
- Tests implemented post-development
- Focus on component rendering and user interactions
- Test files covering core components

LLM-Assisted:

- Vitest + React Testing Library
- Initial tests generated had quality issues
- Required significant manual refinement
- 5 test files with improved mocking strategies
- Generated tests provided good scaffolding despite needing work

7.7 Code Quality & Security

7.7.1 Code Quality Tools

Table 7.5: Code Quality Tools Comparison

Tool	Traditional	LLM-Assisted
Pylint		× (Django-aware, non-blocking)
Bandit		× (Security scanning, low-low threshold)
ESLint	× (Basic)	× (Full configuration)
Code Coverage		× (pytest-cov)

LLM Security Implementation:

- Bandit scans for SQL injection, hardcoded credentials, weak crypto
- Configured with low-low severity (scan everything, alert on high severity)
- Non-blocking integration (reports issues without failing pipeline)
- Particularly valuable for LLM-generated code which may introduce security anti-patterns

7.7.2 Authentication & Security Features

Traditional:

- Token-based authentication
- CORS headers configured for localhost development
- Password hashing with Django defaults
- Server-side validation for all inputs
- No automated security scanning

LLM-Assisted:

- JWT authentication (more modern, stateless)
- Bandit security scanner in CI/CD pipeline
- Environment variable management with `python-decouple`
- Health check endpoint (`/api/health/`) for monitoring
- JSON logging for production debugging
- Same password hashing and input validation standards

7.8 Infrastructure & DevOps

7.8.1 Docker Configuration

Traditional:

- Single-stage Dockerfile
- Basic Docker Compose for local development
- PostgreSQL service with health checks
- Simple environment variable configuration
- Development-focused setup

LLM-Assisted:

- Multi-stage Dockerfile (frontend build, backend deps, production)
- Production-ready configuration with Gunicorn
- Comprehensive health checks for all services
- Environment-based configuration with fallbacks
- Separate Dockerfiles optimized for each service
- Build argument handling for frontend API URL

7.8.2 CI/CD Pipeline Architecture

Pipeline Comparison Insights:

- **Complexity:** Traditional: straightforward 2-job setup; LLM: sophisticated 5-job workflow
- **Quality Gates:** LLM adds Pylint and Bandit for code quality and security
- **Docker Integration:** LLM validates Docker configs before deployment
- **Deployment Strategy:** LLM implements "Build Once" principle with registry-based deployment
- **Caching:** LLM utilizes GitHub Actions cache for Docker layers

7.9 Development Metrics

7.9.1 Time Investment Analysis

Note: Estimates based on project timeline and team reports

Table 7.6: Estimated Development Time Distribution

Phase	**Traditional**	**LLM-Assisted**
Setup & Infrastructure	2–5h	5–8h
Backend Development	180–200h	10–15h
Frontend Development	120–140h	15–20h
Testing	3–5h	2–3h
CI/CD Setup	5–10h	6–8h
Documentation	5–10h	1–2h
Debugging	5–8h	5–10h
Total	**320–378h**	**44–66h**

Team Structure Impact:

- **Traditional:** 5 active developers (shared workload)
- **LLM-Assisted:** Primarily 1 developer with LLM assistance
- **Total Team Effort:** Traditional: 320h across team; LLM: 44h for main developer

7.9.2 Productivity Insights

Traditional Advantages:

- Faster for well-understood, straightforward features
- Immediate code understanding and context

- Lower overhead (no prompt engineering)
- Natural team collaboration and knowledge sharing
- Debugging is direct with full context awareness

LLM-Assisted Advantages:

- Rapid test generation (368% more tests in less time)
- Better architectural structure (5 Django apps vs 1)
- More comprehensive CI/CD (5 jobs vs 2)
- Systematic documentation via prompt records
- Faster boilerplate generation
- Individual productivity multiplier

7.9.3 Development Velocity Patterns

Traditional:

- Steady, linear progression
- Velocity improves over time
- Consistent output across team members
- More time in early phases for architecture decisions

LLM-Assisted:

- Rapid initial progress for scaffold and boilerplate
- Slower for complex logic requiring multiple iterations
- Variable velocity based on prompt quality and LLM performance
- More time in debugging and refinement phases
- Context loss created velocity dips (required thread resets)

7.10 Key Findings & Insights

7.10.1 What Worked Well

Traditional:

- Direct control and complete understanding of codebase
- Faster for simple, well-defined features
- Lower overhead and simpler workflow

- Effective team collaboration and code reviews
- Natural learning through hands-on development

LLM-Assisted:

- Comprehensive automated test coverage (103 tests vs 22)
- Superior modular architecture (5 specialized Django apps)
- Advanced CI/CD pipeline with security scanning
- Systematic documentation through 25 recorded prompts
- Rapid generation of boilerplate and repetitive code
- Better adherence to framework best practices (Django app structure)

7.10.2 What Didn't Work Well

Traditional:

- Less modular architecture (monolithic single app)
- Lower test coverage
- Less sophisticated CI/CD
- More manual effort for boilerplate code
- Limited systematic documentation

LLM-Assisted:

- Frontend code quality issues requiring manual refinement
- Context management challenges (thread length degradation)
- More debugging iterations needed
- Prompt engineering learning curve
- Directory structure confusion after context loss
- Risk of over-reliance without proper code review
- CSS and styling required significant manual work

7.10.3 Critical Success Factors

Traditional Development:

1. Strong team communication and coordination
2. Clear requirements and architectural decisions
3. Consistent code review practices
4. Hands-on debugging skills
5. Domain knowledge and framework familiarity

LLM-Assisted Development:

1. Structured prompt engineering approach
2. Thorough code review of generated code
3. Automated testing as quality gate
4. Iterative refinement mindset
5. Clear project structure documentation
6. Security scanning integration
7. Acceptance of debugging time investment

7.11 Recommendations

7.11.1 When to Use Traditional Approach

- Simple, well-understood features with clear requirements
- Critical security-sensitive code requiring manual verification
- Team has strong domain expertise in the technology stack
- Projects with tight deadlines and minimal scope changes
- Learning-focused environments
- Complex business logic requiring deep understanding

7.11.2 When to Use LLM-Assisted Approach

- Repetitive CRUD operations and boilerplate generation
- Test scaffolding and comprehensive test coverage goals
- CI/CD pipeline configuration and DevOps automation
- Documentation generation and template creation
- Learning new frameworks or exploring architectural patterns
- Projects prioritizing comprehensive testing
- Solo development with limited time resources
- When systematic prompt documentation adds value

7.11.3 Optimal Hybrid Strategy

Based on both implementations, the most effective approach combines strengths of both methodologies:
Use LLM for:

- Initial project scaffold and directory structure
- Boilerplate code generation (models, serializers, basic views)
- Test scaffolding and comprehensive test suites
- CI/CD pipeline configuration
- Documentation templates and API documentation
- Repetitive code patterns and CRUD operations
- Code refactoring suggestions

Use Manual Development for:

- Critical business logic implementation
- Security-sensitive authentication and authorization code
- Complex algorithms (e.g., recommendation engine logic)
- Performance-critical code optimization
- Architecture decisions and system design
- Frontend styling and UX refinement (LLM struggles here)
- Integration of complex third-party services

Always (Regardless of Approach):

- Conduct thorough code review of all generated/written code
- Run automated tests with sufficient coverage
- Perform security scanning (especially for LLM-generated code)
- Execute manual testing for critical user flows
- Document architectural decisions and trade-offs
- Maintain clear git commit history
- Validate against requirements before deployment

7.12 Conclusion

Both approaches successfully achieved the same functional goals through different paths:
Traditional Approach:

- Offers direct control and faster development for simple features
- Requires more manual effort for testing, infrastructure, and boilerplate
- Results in adequate but less modular architecture
- Better for learning and team collaboration
- Total team effort: 320 person-hours across 5 developers

LLM-Assisted Approach:

- Provides better architecture, comprehensive testing, and advanced CI/CD
- Requires skilled prompt engineering and more debugging iterations
- Achieves superior code organization and test coverage
- Multiplies individual developer productivity
- Total effort: 30 hours for primary developer with LLM assistance

Key Takeaway: LLMs are powerful tools that can significantly accelerate development and improve code quality when used appropriately. They excel at generating boilerplate, tests, and infrastructure code, but require skilled prompt engineering, thorough code review, and understanding of when manual development is more appropriate. The optimal strategy is a hybrid approach: leveraging LLMs for repetitive tasks and scaffolding while maintaining manual control for critical business logic and security-sensitive code.

Future Implications: As LLMs improve and coding agents gain better context awareness, many of the current challenges (context management, directory structure understanding, code quality consistency) may diminish. However, the fundamental need for code review, testing, and architectural judgment will remain essential, making the hybrid approach likely the sustainable long-term strategy for professional software development.

Chapter 8

Conclusion

8.1 Project Summary

This project successfully developed FilmHub, a comprehensive movie recommendation system, while exploring two distinct development methodologies: traditional manual development and AI-assisted development using Large Language Models. The system was implemented as a full-stack web application with Django REST Framework backend, React frontend, and PostgreSQL database, demonstrating the complete Software Development Lifecycle from requirements analysis through deployment.

The comparative analysis between traditional and AI-assisted approaches revealed significant differences in development velocity, code quality, and workflow efficiency. The traditional implementation, developed by a team of five members over three months, resulted in a robust system with 22 comprehensive tests, 276 commits, and extensive peer review through 20 pull requests. In contrast, the AI-assisted version was completed by a single developer in approximately 30 hours, demonstrating the potential of LLMs to accelerate certain development tasks.

8.2 Key Findings

8.2.1 Development Velocity and Efficiency

The AI-assisted approach demonstrated remarkable efficiency gains in specific areas:

- **Rapid Prototyping:** Initial project structure, boilerplate code, and basic CRUD operations were generated significantly faster using LLM assistance, reducing setup time from days to hours.
- **Reduced Context Switching:** A single developer could work across the full stack more efficiently with LLM support, eliminating coordination overhead present in the traditional team-based approach.
- **Accelerated Implementation:** Features such as JWT authentication, watch list management, and user profile systems were implemented more quickly with AI-generated code templates and suggestions.

However, the traditional approach provided benefits in other dimensions:

- **Collaborative Learning:** Team-based development facilitated knowledge sharing, peer learning, and collective problem-solving that enhanced overall team competency.
- **Code Review Benefits:** The 20 pull requests in the traditional repository ensured higher code quality through systematic peer review, catching potential issues early.
- **Distributed Expertise:** Specialized roles allowed team members to develop deep expertise in specific areas (frontend, backend, DevOps), resulting in more sophisticated solutions in their respective domains.

8.2.2 Code Quality and Testing

The traditional implementation demonstrated stronger emphasis on testing and quality assurance:

- Comprehensive test suite with 22 unit and integration tests
- Systematic validation of business logic and edge cases
- Robust error handling developed through iterative debugging sessions
- Well-documented code reflecting team knowledge accumulation

The AI-assisted version, while functional, had less extensive testing coverage, highlighting a common challenge when using LLMs: the tendency to prioritize feature implementation over comprehensive testing strategies.

8.2.3 Architecture and Design Decisions

Both implementations adopted similar architectural patterns (three-tier client-server monolith), but with notable differences:

- **Authentication:** Traditional version used Django REST Framework's token authentication, while AI-assisted version implemented JWT, demonstrating LLM awareness of modern best practices.
- **Data Management:** Traditional version fetched movies on-demand from TMDB API, while AI-assisted version pre-populated database with 300 movies, reflecting different optimization strategies.
- **Code Organization:** Traditional version evolved a more granular structure through refactoring, while AI-assisted version maintained simpler organization suitable for rapid development.

8.3 DevOps and Deployment

Both versions successfully implemented modern DevOps practices:

- Containerization using Docker for consistent development and production environments
- CI/CD pipelines configured with GitHub Actions for automated testing and deployment
- Successful deployment to cloud platforms (Render) with proper environment configuration
- Implementation of best practices for security, including environment variable management and CORS configuration

The deployment process validated the portability and production-readiness of both implementations, confirming that AI-assisted development can produce deployable applications when properly guided.

8.4 Lessons Learned

8.4.1 Technical Insights

- **LLM Strengths:** Excellent for boilerplate generation, API integration, common patterns, and rapid prototyping. Particularly effective for well-established technologies with extensive training data.
- **LLM Limitations:** Less reliable for complex business logic, novel problem-solving, comprehensive testing strategies, and nuanced architectural decisions that require deep domain understanding.
- **Hybrid Approach Value:** The optimal workflow likely combines LLM assistance for routine tasks with human oversight for critical decisions, quality assurance, and creative problem-solving.
- **Full-Stack Considerations:** Modern web development frameworks (Django, React) with strong conventions and documentation work particularly well with LLM assistance, as the models have extensive training on these patterns.

8.4.2 Process Insights

- **Team Dynamics:** Traditional development fostered stronger team cohesion, knowledge sharing, and collective problem-solving capabilities that extend beyond the immediate project.
- **Communication Overhead:** While team coordination required significant effort (20+ hours in meetings and lab sessions per member), it ensured alignment and prevented major architectural mismatches.

- **Learning Outcomes:** Traditional development provided deeper understanding of underlying technologies and design patterns, while AI-assisted development emphasized prompt engineering and output validation skills.
- **Maintenance Considerations:** Code produced through collaborative traditional development may be more maintainable long-term due to shared team understanding and comprehensive documentation.

8.5 Recommendations for Future Projects

Based on our experience, we offer the following recommendations for teams considering AI-assisted development:

8.5.1 When to Use AI Assistance

- **Prototyping and MVPs:** LLMs excel at quickly building functional prototypes to validate ideas and gather early user feedback.
- **Boilerplate and Setup:** Use AI for project initialization, configuration files, and standard CRUD operations to save significant time.
- **Learning New Technologies:** LLMs can accelerate learning curves by providing working examples and explaining unfamiliar patterns.
- **Solo Development:** Individual developers can leverage LLMs to simulate having specialized team members for areas outside their expertise.

8.5.2 When to Emphasize Traditional Approaches

- **Complex Business Logic:** Critical algorithms and domain-specific logic benefit from careful human design and validation.
- **Security-Critical Components:** Authentication, authorization, and data protection require thorough human review regardless of generation method.
- **Long-Term Maintenance:** Projects with extended lifespans benefit from the deep understanding that comes from manual development.
- **Team Learning Objectives:** Educational contexts should balance AI assistance with hands-on coding to ensure skill development.

8.5.3 Best Practices for Hybrid Development

- **Validate AI Output:** Always review, test, and understand LLM-generated code before integration. Never blindly accept suggestions.
- **Maintain Test Coverage:** Compensate for LLM testing gaps by manually writing comprehensive test suites.
- **Document Decisions:** Record architectural decisions and rationale, whether code is AI-generated or manually written.

- **Version Control Discipline:** Use clear commit messages and branching strategies regardless of development approach.
- **Code Review Process:** Implement systematic code review even for AI-generated code to catch subtle issues and ensure quality.

8.6 Project Outcomes

The FilmHub project successfully achieved its primary objectives:

1. **Functional System Delivery:** Both traditional and AI-assisted versions are fully operational, deployed, and accessible online, providing meaningful movie recommendations to users.
2. **Comparative Analysis:** Systematic comparison of development methodologies provided valuable empirical data on the effectiveness of AI-assisted development in real-world scenarios.
3. **DevOps Implementation:** Successfully configured and deployed CI/CD pipelines, demonstrating modern software engineering practices.
4. **Team Development:** All team members gained substantial experience in full-stack development, version control, DevOps practices, and software project management.
5. **Documentation:** Comprehensive documentation of system architecture, implementation decisions, and comparative findings provides a valuable reference for future projects.

The total team effort of 470 person-hours resulted in a production-ready application that demonstrates both technical competence and practical understanding of modern software engineering principles.

8.7 Future Work

Several directions for future enhancement and research emerged from this project:

8.7.1 Technical Enhancements

- **Advanced Recommendation Algorithms:** Implement collaborative filtering or matrix factorization techniques to improve recommendation quality beyond simple genre and keyword matching.
- **Real-Time Updates:** Add WebSocket support for real-time notifications when new recommendations become available.
- **Social Features:** Enable users to share recommendations, follow other users, and create public watch lists.

- **Performance Optimization:** Implement caching strategies for TMDB API responses and database query optimization for improved scalability.
- **Mobile Applications:** Develop native mobile applications for iOS and Android to complement the web interface.

8.7.2 Research Directions

- **Long-Term Maintenance Study:** Track both implementations over 6-12 months to compare maintenance effort, bug frequency, and feature addition complexity.
- **Code Quality Metrics:** Apply automated code quality tools (SonarQube, CodeClimate) to quantify differences in code complexity, duplication, and maintainability.
- **User Experience Testing:** Conduct user studies to determine if development methodology affects end-user experience and application usability.
- **Larger Team Comparison:** Expand the study to larger teams (10+ members) to assess how AI assistance scales with team size.
- **Different LLM Comparison:** Compare multiple LLM tools (GitHub Copilot, Amazon CodeWhisperer, Claude, GPT-4) to identify strengths and weaknesses of each.

8.8 Final Reflection

The development of FilmHub provided invaluable insights into the evolving landscape of software engineering. The comparative analysis between traditional and AI-assisted approaches revealed that neither methodology is universally superior—rather, each has distinct advantages that make them suitable for different contexts and objectives.

AI-assisted development, exemplified by the LLM-based implementation, demonstrates tremendous potential for accelerating certain aspects of software development, particularly in the areas of code generation, boilerplate creation, and rapid prototyping. A single developer leveraging LLM tools completed in approximately 30 hours what traditionally required a team of five over several months of coordinated effort. This efficiency gain is remarkable and suggests that AI assistance will become an increasingly important tool in the software engineering toolkit.

However, this efficiency comes with important caveats. The traditional team-based approach fostered deeper learning, more robust testing practices, better documentation, and stronger collaborative skills—all of which are crucial for long-term career development and maintaining complex software systems. The 20 pull requests and systematic code review process in the traditional repository reflected a level of quality assurance and knowledge sharing that the AI-assisted approach did not fully replicate.

Perhaps most importantly, this project demonstrated that the future of software engineering is not about choosing between human developers and AI tools, but rather about finding the optimal integration of both. The most effective approach likely involves using LLMs to handle routine, well-understood tasks while reserving human creativity, judgment, and expertise for complex problem-solving, architectural decisions, and quality assurance.

As AI tools continue to evolve and improve, software engineers must develop new skills: the ability to effectively prompt and guide AI systems, critically evaluate AI-generated code, and make informed decisions about when to leverage automation versus when to rely on human expertise. This project has provided our team with early experience in this emerging skillset, preparing us for a software engineering landscape where AI assistance is not just a novelty but a fundamental component of professional practice.

The FilmHub project stands as both a functional movie recommendation system and a learning experience that has equipped our team with practical knowledge of modern development practices, emerging AI tools, and the critical thinking skills necessary to navigate an evolving technological landscape. The lessons learned and skills developed throughout this project will serve as a foundation for our future endeavors in software engineering.